

Multimodal and Generative Models: Diffusion, Contrastive Learning, Tokens and Speech

Unit 7 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): the forward process and the training objective	6
3.3 Tier 3 (medium-hard): sampling, guidance, contrastive learning	6
3.3.1 DDPM and DDIM sampling	6
3.3.2 Conditioning and classifier-free guidance	8
3.3.3 CLIP: contrastive learning and zero-shot classification	9
3.4 Tier 4 (hard): tokens, vision-language models, speech, video	10
3.4.1 Images as tokens	10
3.4.2 Vision-language models	10
3.4.3 Speech: features, speech-to-text, text-to-speech	11
3.4.4 Video: a quick estimate	12
3.5 Tier 5 (hardest): open questions	12
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures	12
5. Analyze	13

5.1 Limits	13
5.2 Which term dominates	13
5.3 Sensitivity	13
5.4 Failure modes	14
5.5 Uncertainty	14
6. Evaluate	14
7. Create	14
8. Visualization lesson	15
9. Problem set	15
10. Retrieval practice and recap	15
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. derive the **forward diffusion** formula and the noise-prediction loss, and write the DDPM and DDIM sampling steps;
2. explain **classifier-free guidance** and the trade-off it makes;
3. explain **contrastive (CLIP-style) learning** and how it gives zero-shot classification;
4. count **tokens for images, video and audio**, and say why attention makes long video expensive;
5. describe the **speech pipeline** (log-mel features, speech-to-text, text-to-speech) and compute the **word error rate**.

- **Prerequisites:** Unit 1 (the diffusion idea in one paragraph, the VAE and GAN), Unit 2 (attention cost grows with the square of the number of tokens, the Transformer block), Unit 3 (embeddings, cosine similarity), Unit 4 (paired comparison). Gaussian random variables: sums of independent Gaussians are Gaussian, with the variances added.
- **Difficulty ladder used:** easy → medium → medium-hard → hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 7 figures, the module `codes/multimodal/multimodal_np/` (a conditional diffusion model with DDPM and DDIM samplers and classifier-free guidance, a trained toy model, a CLIP-style contrastive model, patch and token counts, STFT and log-mel features, word error rate; 13 tests), the *Diffusion denoiser* widget (which runs the actual trained network in your browser) and the *Tokens* widget, and a Colab notebook for the practical “a small multimodal application”.

- **Source anchors:** Ho et al. 2020, arXiv:2006.11239 (DDPM: $T = 1000$, β_t from 10^{-4} to 0.02; Algorithm 1 trains by minimising $\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$; Algorithm 2 samples with $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta) + \sigma_t z$); Song et al. 2020, arXiv:2010.02502 (DDIM: deterministic generative processes that produce samples “10× to 50× faster”); Ho and Salimans 2022, arXiv:2207.12598 (classifier-free guidance, $\tilde{\epsilon} = (1 + w)\epsilon_\theta(z, c) - w\epsilon_\theta(z)$, “a trade-off between sample quality and diversity”); Rombach et al. 2021, arXiv:2112.10752 (latent diffusion: diffusion in the latent space of an autoencoder, with cross-attention for conditioning); Radford et al. 2021, arXiv:2103.00020 (CLIP: predicting which caption goes with which image on 400 million image-text pairs, then zero-shot transfer); Dosovitskiy et al. 2020, arXiv:2010.11929 (ViT: an image $x \in \mathbb{R}^{H \times W \times C}$ reshaped into $N = HW/P^2$ flattened patches); Radford et al. 2022, arXiv:2212.04356 (Whisper: audio resampled to 16 kHz, 80-channel log-magnitude Mel spectrogram on 25 ms windows with a 10 ms stride, 30-second segments). All IDs and the quoted statements were checked in the PDFs on the day of writing.
- **What is measured here, and what is not.** The diffusion and contrastive experiments run on **small 2-D and toy-feature data sets with NumPy**; they show mechanisms and orders of magnitude, not the quality of real image or speech systems. No image, speech or vision-language model was run on this machine; the practical with real models is a Colab notebook that was **not executed** while writing this document.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
multimodal model		reads or writes more than one kind of data	3.1	new
forward diffusion process	$\bar{\alpha}_t$	gradually adds Gaussian noise to data	3.2	new
noise-prediction network	ϵ_θ	predicts the noise that was added	3.2	new
DDPM sampling		reverse process with a small random kick at each of T steps	3.3	new
DDIM sampling		deterministic reverse process over fewer steps	3.3	new
classifier-free guidance	w	extrapolate from the unconditional to the conditional prediction	3.3	new
latent diffusion		diffusion in an autoencoder’s compressed space	3.3	new
contrastive learning (CLIP)	τ	pull matching image-text pairs together, push others apart	3.3	new
zero-shot classification		classify by similarity to text prompts of the class names	3.3	new
vision-language model		image tokens from a vision encoder read by a language model	3.4	new
image patch token	P	one flattened square patch of the image	3.4	new
log-mel spectrogram		energy in 80 frequency bands per 10 ms frame	3.4	new

Term	Symbol	One-line meaning	Where it appears	New / known
speech-to-text, text-to-speech		audio to words, words to audio	3.4	new
word error rate (WER)		word-level edit distance divided by the reference length	3.4	new
embedding, cosine similarity		vector for a text or image; angle-based closeness	3.3	known

Terms *used but not taught here*: vocoders and neural audio codecs, image quality metrics such as FID, video diffusion models (outlined only), preference tuning of image models.

2. Remember

Diffusion model

A **diffusion model** learns to reverse a gradual corruption. The **forward process** adds a little Gaussian noise at each of T steps, $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t$. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$ the noisy sample can be written directly from the clean one:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (1)$$

A network $\epsilon_\theta(x_t, t)$ is trained to **predict the noise** ϵ ; generation starts from pure noise and removes it step by step.

Contrastive learning (CLIP)

Embed N images and their N captions with two encoders, normalise, and form the similarity matrix $S_{ij} = \cos(z_i^x, z_j^t) / \tau$. Train so that the diagonal (the true pairs) is the largest entry of each row *and* each column: a cross-entropy over rows plus one over columns. τ is a **temperature**.

Tokens for other media

An image of $H \times W$ pixels cut into $P \times P$ patches gives $N = HW/P^2$ tokens, each a flattened patch of $P^2 C$ numbers. Audio at 16 kHz is turned into a **log-mel spectrogram**: one 80-number frame every 10 ms.

Word error rate

$\text{WER} = (S + D + I) / N_{\text{ref}}$: the minimum number of word substitutions, deletions and insertions that turn the hypothesis into the reference, divided by the number of reference words.

Memory hooks.

- *Diffusion = restoring a painting buried in sand*: learn to brush away one thin layer at a time, then start from a field of sand and “restore” a painting that never existed.

- *Guidance = turning up the volume on the prompt*: louder obedience, less variety, and at the extreme, distortion.
- *CLIP = a matching game with a million cards*: the model learns what pairs with what; zero-shot classification is playing the game with the class names as the captions.
- *Tokens: pictures are paragraphs of patches; sound is a paragraph of frames*.

Recall quiz (answers in Section 10).

1. What does the noise-prediction network take as input and what must it output?
2. Why does DDIM allow far fewer steps than DDPM?
3. What does classifier-free guidance weight $w = 0$ give?
4. How does CLIP classify an image without a classifier trained on labels?
5. By what factor does the attention cost grow when the image side doubles and the patch size is kept?

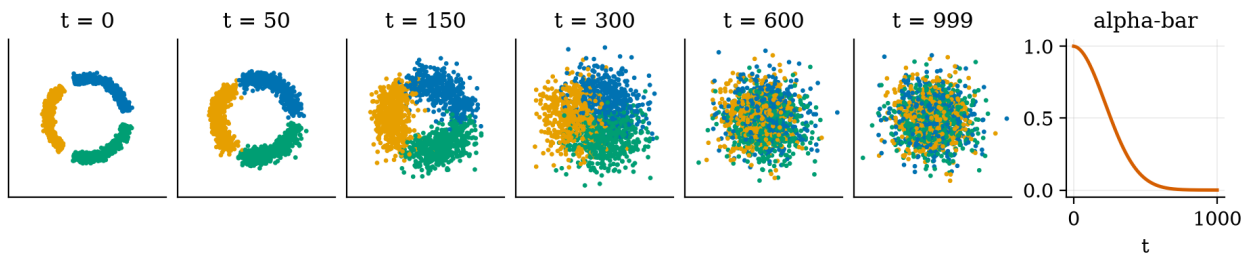
3. Understand: the derivation

3.1 Tier 1 (easy): intuition and a picture

Text models (Units 2 and 3) write one token at a time. Images, video and sound can be turned into tokens too (Section 3.4), but *generating* an image token by token is awkward. **Diffusion** does it differently, in two parts.

1. **Corrupt.** Take a real sample and add noise in many small steps until nothing is left but noise. This direction needs no learning: it is a fixed recipe.
2. **Restore.** Train a network to undo one small step. Then to *generate*, start from pure noise and apply the network again and again until a sample appears.

To make the ideas visible I use the smallest possible “image”: a point in the plane. The data are three noisy arcs of a circle (class 0, 1, 2). The figure shows the corruption.



Forward diffusion on 1500 points from three arcs (colours are the classes) at steps $t = 0, 50, 150, 300, 600, 999$, and the schedule $\bar{\alpha}_t$ (right). $\bar{\alpha}_t$ falls from 0.9999 to 4×10^{-5} : the share of signal amplitude $\sqrt{\bar{\alpha}_t}$ is 0.985 at $t = 50$, 0.628 at $t = 300$, 0.16 at $t = 600$ and 0.006 at $t = 999$.

By $t = 300$ the arcs are blurred; by $t = 600$ they are almost gone (16% of the signal amplitude is left), and at $t = 999$ the cloud is pure Gaussian noise (0.6%), which is exactly the starting point for generation. The schedule is the one of Ho et al. (2020): $T = 1000$ steps with β_t rising linearly from 10^{-4} to 0.02.

3.2 Tier 2 (medium): the forward process and the training objective

Closed form of the forward process. Start from one step, $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t$ with $\beta_t = 1 - \alpha_t$, and substitute the previous step, $x_{t-1} = \sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \underbrace{\sqrt{\alpha_t \beta_{t-1}} \epsilon_{t-1} + \sqrt{\beta_t} \epsilon_t}_{\text{Gaussian with variance } \alpha_t(1-\alpha_{t-1})+(1-\alpha_t)=1-\alpha_t\alpha_{t-1}}.$$

(the two noise terms are independent Gaussians, so their variances add). Repeating down to x_0 gives (1). The test `test_forward_noising_variance_and_mean` checks the mean and variance numerically.

Training. Pick a data point x_0 , a random step t , noise ϵ , form x_t by (1), and teach the network to predict ϵ (Algorithm 1 of Ho et al.):

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, t)\|^2. \quad (2)$$

This is plain regression, with a target that is easy to generate. The network ϵ_θ here is a small multilayer perceptron with hand-written backpropagation (Unit 1) whose input is the noisy point, a sinusoidal embedding of t (the same device as positional encoding in Unit 2) and a one-hot class label; the label is replaced by a “null” label with probability 0.1 during training, which is needed for guidance (Section 3.3.2). I trained it for 30,000 steps (batch 512, 199 seconds on a laptop CPU); the final loss was 0.167.

What the network finds hard. Predicting the noise is easy when there is a lot of it: at $t = 600$, $x_t \approx 0.16 x_0 + 0.99 \epsilon$, so $\epsilon \approx x_t$. It is hard when there is little: at $t = 100$ the network must know where the *data* lie in order to say which part of x_t is noise. An exploratory run with a smaller model (2000 steps, 48 hidden units; the module’s test asserts the same ordering with looser thresholds) shows this: the correlation between predicted and true noise is 0.71 at $t = 100$, 0.87 at $t = 300$ and 0.99 at $t = 600$.

3.3 Tier 3 (medium-hard): sampling, guidance, contrastive learning

3.3.1 DDPM and DDIM sampling

The reverse step. The true posterior of the previous state, given the current state *and* the clean sample, is Gaussian with mean

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t.$$

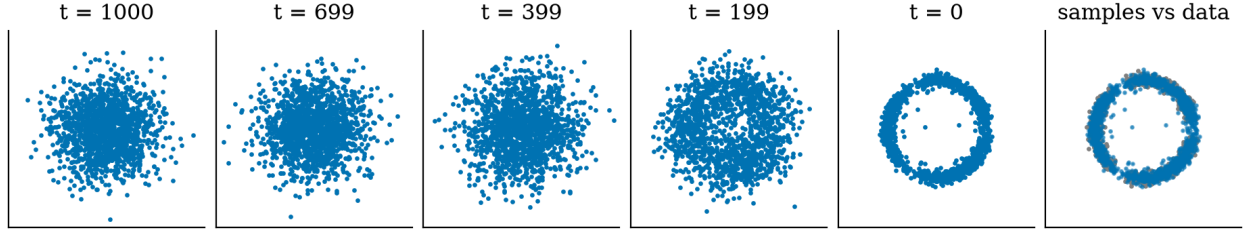
We do not know x_0 , but (1) gives $x_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) / \sqrt{\alpha_t}$. Substitute, using $\sqrt{\alpha_{t-1}} / \sqrt{\alpha_t} = 1 / \sqrt{\alpha_t}$. The coefficient of ϵ is $-\beta_t / (\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t})$, and the coefficient of x_t is

$$\frac{\beta_t + \alpha_t (1 - \bar{\alpha}_{t-1})}{\sqrt{\alpha_t} (1 - \bar{\alpha}_t)} = \frac{1 - \alpha_t + \alpha_t - \bar{\alpha}_t}{\sqrt{\alpha_t} (1 - \bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}},$$

using $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. So the mean is $\frac{1}{\sqrt{\alpha_t}} (x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon)$, and replacing the true ϵ by the network’s prediction gives **Algorithm 2** of Ho et al.:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad \sigma_t^2 = \beta_t, \quad z \sim \mathcal{N}(0, I) \quad (z = 0 \text{ at the last step}). \quad (3)$$

The test `test_ddpm_reverse_mean_equals_true_posterior_mean` checks this algebra to 10^{-10} .



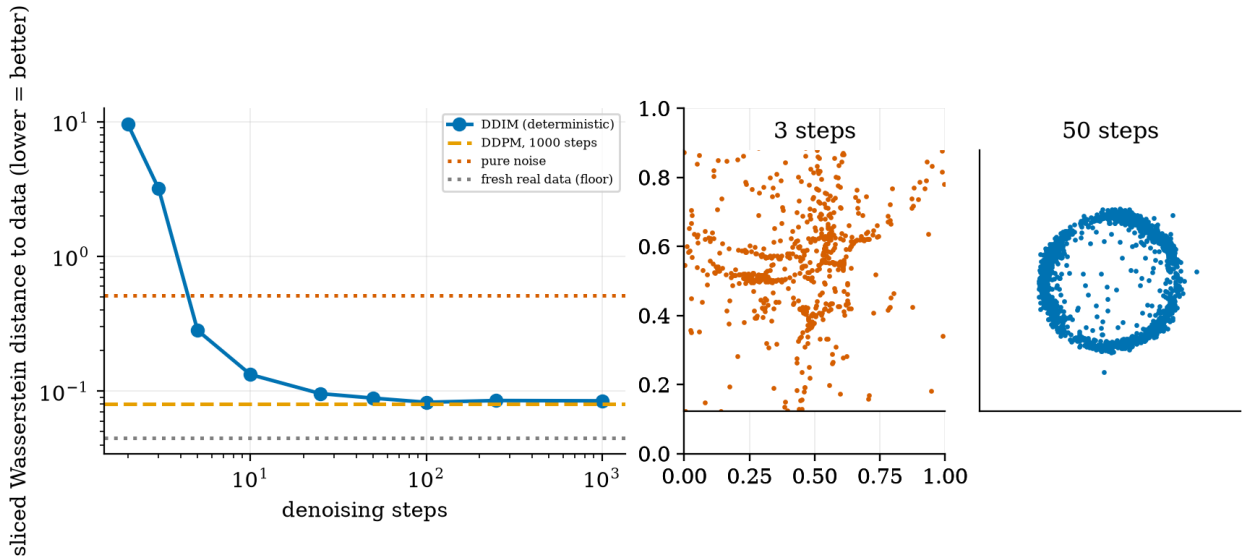
The reverse path with 1000 DDPM steps: states at $t = 999, 699, 399, 199, 0$ from pure noise (left) to samples (fifth panel), and the final samples (blue) over the real data (grey) in the last panel. The sliced Wasserstein distance to the data is 0.081 for the samples, 0.512 for pure noise and 0.045 for a fresh batch of real data.

The **sliced Wasserstein distance** I use as a quality measure projects both clouds onto many random directions, sorts the projections, and averages the absolute differences: it is 0 for identical clouds and grows as they differ. Three reference values calibrate it: pure noise 0.512 (bad), samples 0.081, and a fresh batch of *real* data 0.045 (the floor: even real data differ from other real data by this much). The model’s samples are close to the floor but not at it.

DDIM. 1000 network evaluations per sample is slow. Song et al. (2020) show that the same trained network supports a *deterministic* sampler that visits only a subset of the steps. At step t the network predicts the clean sample, $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta) / \sqrt{\bar{\alpha}_t}$, and the next state, at the next chosen step t' , is

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}} \epsilon_\theta(x_t, t). \quad (4)$$

At the final step $\bar{\alpha}_{t'} = 1$, so the last output is $\hat{x}_0$ itself. (This is their sampler with the extra noise set to zero.) The test `test_ddim_with_a_perfect_denoiser_recovers_x0_for_any_number_of_steps` checks a property that any correct implementation must have: with a *perfect* noise predictor, DDIM returns the original x_0 exactly, whether it uses 1, 5, 50 or 1000 steps. Song et al. report 10 to 50 times faster sampling at high quality; Figure 3 measures it on my toy.



Left: sliced Wasserstein distance to the data (log-log) against the number of DDIM steps 2, 3, 5, 10, 25, 50, 100, 250, 1000: 9.62, 3.18, 0.282, 0.133, 0.096, 0.089, 0.083, 0.085, 0.085, with DDPM-1000 at 0.081, pure noise at 0.512 and fresh real data at 0.045. Right: the 3-step and 50-step samples.

Reading Figure 3: from about **25 steps** DDIM is within 0.015 of the 1000-step DDPM sampler; at

10 steps it is 0.133 (usable but visibly worse); at 5 steps 0.28; and at **3 steps (3.18) and 2 steps (9.62) it is far worse than pure noise (0.512)**. Why? The first DDIM step jumps from $t = 999$ to a much smaller t' and computes $\hat{x}_0$ by dividing by $\sqrt{\alpha_{999}} = 0.0064$, which multiplies any error of the network by about $1/0.0064 \approx 157$. With few steps there is no chance to correct this; with many steps each jump is short and errors are fixed on the way. (That DDIM can use fewer steps is thus a statement about *this* trade-off, not a free lunch.)

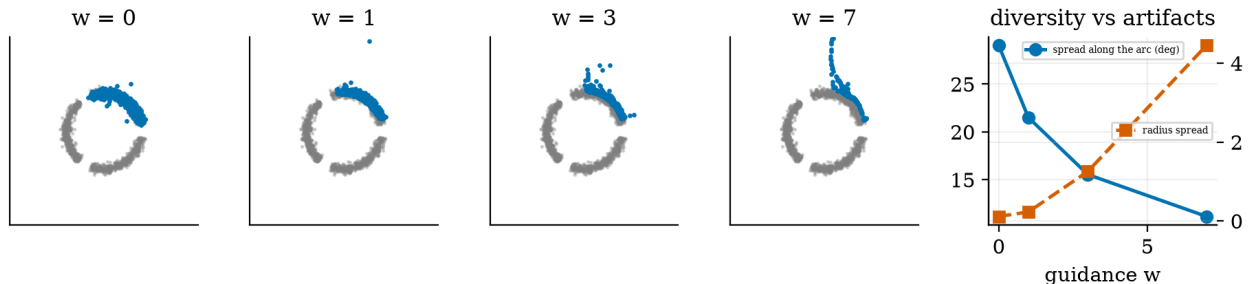
3.3.2 Conditioning and classifier-free guidance

To generate “an arc of class 1”, feed the label to the network (as I did): $\epsilon_\theta(x_t, t, c)$. For text-to-image, c is a text embedding. **Latent diffusion** (Rombach et al.) makes this affordable: it runs the diffusion in the compressed latent space of a pre-trained autoencoder, so each step works on far fewer numbers (if the autoencoder shrinks each side by a factor $f = 8$, the grid has $f^2 = 64$ times fewer positions), and it injects the text through **cross-attention** layers (Unit 2) inside the denoiser.

Classifier-free guidance (Ho and Salimans) trains one network for both the conditional and the unconditional task, by dropping the label at random (here 10% of the time), and at sampling time extrapolates from one to the other:

$$\tilde{\epsilon} = (1 + w) \epsilon_\theta(x_t, t, c) - w \epsilon_\theta(x_t, t) = \epsilon_\theta(x_t, t) + (1 + w) [\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t)]. \quad (5)$$

With $w = 0$ this is the plain conditional prediction; $w > 0$ moves *beyond* it, away from what the model would produce without the condition. The test `test_classifier_free_guidance_formula` checks the formula. Ho and Salimans describe the result as “a trade-off between sample quality and diversity”. Figure 4 measures it on class 0, with DDIM and 50 steps:



Classifier-free guidance for arc 0 with $w = 0, 1, 3, 7$ (blue: samples; grey: the data of all three arcs), and, at the right, the spread of the samples along the arc (blue, degrees) and the spread of their distance from the centre (red). Spread along the arc: 29.1°, 21.5°, 15.5°, 11.2° (the true arc: 28.9°). Radius standard deviation: 0.12, 0.23, 1.26, 4.45; points more than 0.5 from the circle: 0%, 0%, 2%, 7%.

Three honest observations. (1) **Purity did not change**: the share of samples that landed on the right arc was 1.000 at every w , because my three arcs are well separated, so even the plain conditional model already obeys the label; guidance does not help what is not broken. (2) **Variety falls steadily**: the spread along the arc shrinks from the true 29° to 11°, so samples bunch toward the middle of the arc (a “mode-seeking” effect). (3) **Large w breaks the sample**: at $w = 7$ the radius standard deviation rises from 0.12 to 4.45 and 7% of the points fly off the circle: extrapolating the prediction too far leaves the region the model learned. That is the toy version of over-saturated, distorted images at very high guidance scales. The widget lets you see all three.

3.3.3 CLIP: contrastive learning and zero-shot classification

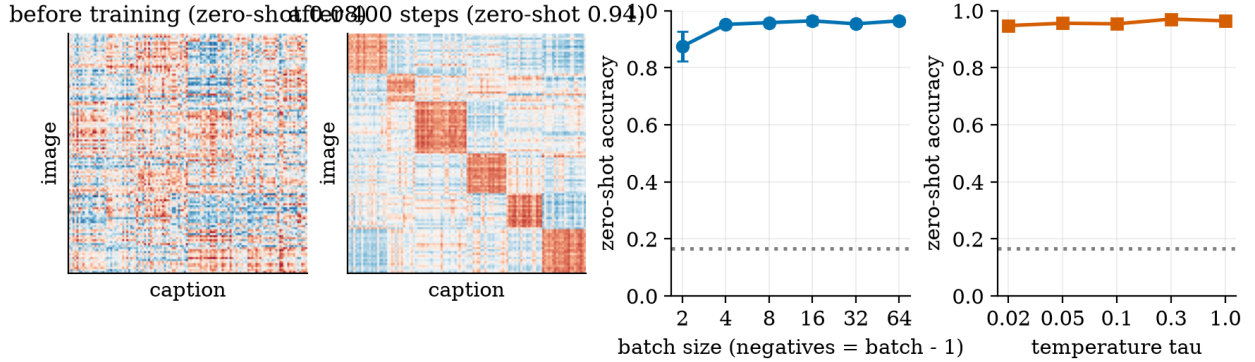
Given a batch of N pairs, two encoders produce unit vectors z_i^x, z_j^t and the similarity matrix $S = Z^x Z^t{}^\top / \tau$. The loss is the average of two cross-entropies, one classifying for each image its caption among the N captions (rows) and one for each caption its image (columns):

$$\begin{aligned}\mathcal{L} &= -\frac{1}{2N} \sum_i [\ell_i^{\text{row}} + \ell_i^{\text{col}}], \\ \ell_i^{\text{row}} &= \ln \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}}, \\ \ell_i^{\text{col}} &= \ln \frac{e^{S_{ii}}}{\sum_j e^{S_{ji}}}.\end{aligned}\tag{6}$$

Radford et al. train this on 400 million image-text pairs. The gradient is the familiar one from Unit 3: with P^r the row-softmax and P^c the column-softmax, $\partial\mathcal{L}/\partial S = \frac{1}{2N}[(P^r - I) + (P^c - I)^\top]$, pushed back through the normalisation and the encoders (the CLIP gradient test in `test_multimodal_np.py` checks the whole chain numerically).

Zero-shot classification. After training, write each class name as a short caption (“a photo of a cat”), embed those prompts, and give an image the label of the most similar prompt. No classifier was trained on labels: the class names are just text.

I built a toy: 6 classes, “image” features that are a class prototype plus noise (24 numbers), and “captions” that are bags of 18 words, each class having 3 synonym words, with each caption holding 2 of its synonyms and one random distractor word. Linear encoders map both to 8 dimensions.



Left two panels: the similarity matrix of 120 test pairs sorted by class, before training (zero-shot accuracy 0.08) and after 400 steps (0.94): blocks along the diagonal mark same-class pairs. Right two panels: zero-shot accuracy against the batch size (2 to 64) and against the temperature τ (0.02 to 1.0), means of 4 seeds with their standard deviations.

Results (chance is $1/6 = 0.167$): before training, image-to-caption retrieval (is the top caption of the right class?) 0.183 and zero-shot accuracy 0.083; after 400 steps with batch 32 and $\tau = 0.1$, 0.950 and 0.942. The loss falls from 5.47 to 2.05. Two details are worth understanding. The start is *above* $\ln 32 = 3.47$ (the loss of random guessing) because at $\tau = 0.1$ random similarities are scaled by 10 and are confidently wrong. The end is *above* 0 because captions of the same class are interchangeable: with 32 pairs and 6 classes, about $32/6 = 5.3$ same-class captions compete with the true one, so even a perfect class-level model has a loss near $\ln 5.3 = 1.7$; the achieved 2.05 is close to that floor. And the toy **contradicts a common intuition**: zero-shot accuracy was 0.875 ± 0.051 at batch 2, then 0.95 to

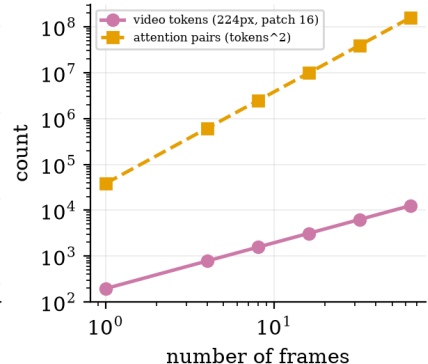
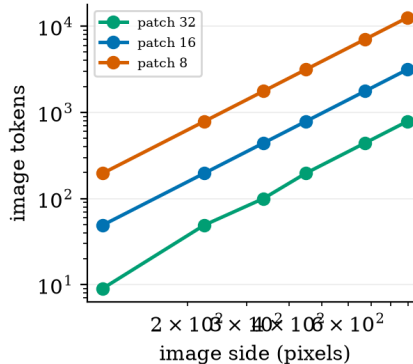
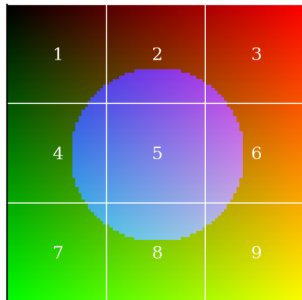
0.97 for every batch of 4 or more, and it hardly depended on τ (0.948 to 0.971). With only 6 classes a handful of negatives is enough. Real CLIP has millions of distinct concepts, where many negatives matter; I did not test that.

3.4 Tier 4 (hard): tokens, vision-language models, speech, video

3.4.1 Images as tokens

Following Dosovitskiy et al., an image $x \in \mathbb{R}^{H \times W \times C}$ is reshaped into $N = HW/P^2$ flattened patches of P^2C numbers; each is mapped by a linear layer to a vector, and the sequence goes through a Transformer like text tokens (Unit 2). `patchify` and `unpatchify` in the module are exact inverses (a test checks it). A 224×224 RGB image with $P = 16$ gives $N = 14^2 = 196$ tokens of $16 \cdot 16 \cdot 3 = 768$ numbers each.

96x96 image, 32x32 patches: 9 tokens



Left: a 96×96 image cut into nine 32×32 patches (each a vector of 3072 numbers). Middle: image tokens against the image side for patch sizes 32, 16 and 8. Right: video tokens (224 pixels, patch 16) and attention pairs (tokens squared) against the number of frames.

Cost. Self-attention compares every token with every other (Unit 2), so its cost grows with N^2 . For $P = 16$ the token counts and the attention cost relative to a 224-pixel image are:

doubling the side multiplies the tokens by 4 and the attention cost by 16.

image side	112	224	336	448	672	896
tokens	49	196	441	784	1764	3136
attention cost vs 224 px	0.06	1	5.1	16	81	256

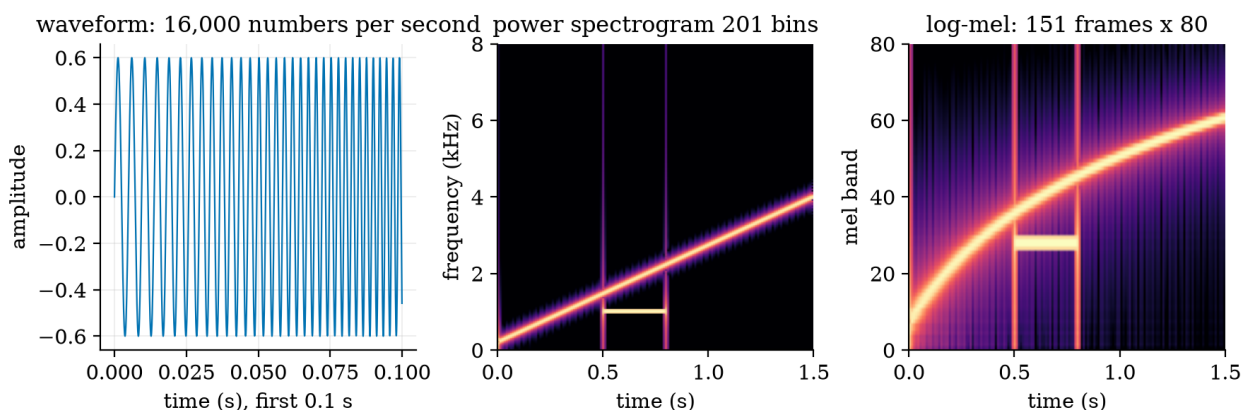
3.4.2 Vision-language models

A **vision-language model** (VLM) connects a vision encoder to a language model: the encoder turns the image into patch tokens; a small “projector” maps them into the language model’s embedding space; and the language model reads them *as if they were words*, together with the text prompt, and writes the answer. Image question answering, captioning and chart reading all work this way. Two consequences from earlier units apply directly: the **image tokens share the context window** (Unit 4’s budget), so one high-resolution image can use thousands of tokens; and everything the language

model knows about text still holds (it can hallucinate about an image as it can about a document, Unit 5). The Colab notebook runs a real VQA model, a CLIP classifier and a speech model.

3.4.3 Speech: features, speech-to-text, text-to-speech

Sound is a waveform: 16,000 numbers per second at 16 kHz. Speech models do not read it directly; they read a **spectrogram**. The short-time Fourier transform cuts the waveform into overlapping windows (25 ms = 400 samples), multiplies each by a Hann window, and takes the magnitude of the Fourier transform; frames start every 10 ms (= 160 samples), so there are 100 frames per second. The frequency axis is then pooled into 80 **mel bands** (equally spaced on the mel scale $m = 2595 \log_{10}(1 + f/700)$, which follows human pitch perception) and the logarithm is taken. This is exactly the recipe Radford et al. (2022) use for Whisper: 16 kHz, 80-channel log-magnitude mel spectrogram, 25 ms windows, 10 ms stride, 30-second segments.



A synthetic 1.5 s signal (a rising 200 to 4000 Hz chirp plus a 1 kHz tone burst between 0.5 and 0.8 s). Left: the first 0.1 s of the waveform. Middle: the power spectrogram (151 frames by 201 frequency bins). Right: the 80-band log-mel features (151 frames by 80 bands). The burst lights mel band 28 (a 1 kHz tone is 1000 of the 2840 mel up to 8 kHz, 35% of the bands), and the chirp climbs from band 21 at 0.2 s to band 57 at 1.3 s.

The numbers: 1.5 s gives 151 frames (the extra one comes from padding the signal so that frames are centred); 30 s gives 3001 frames of 80 numbers. The waveform has 16,000 numbers per second; the log-mel features have $100 \times 80 = 8000$. **Speech-to-text** (Whisper) is an encoder-decoder Transformer (Unit 2) that reads the mel frames and writes text tokens. **Text-to-speech** runs the other way: text $\rightarrow$ acoustic features (such as mel frames) $\rightarrow$ a **vocoder** that turns them into a waveform; I describe it without implementing it.

Word error rate. The module computes WER by word-level edit distance. Hand-checked cases: reference “the cat sat on the mat” against “the cat sit on mat”: one substitution (*sat* $\rightarrow$ *sit*) and one deletion (the second “the”), so $WER = 2/6 = 0.333$; identical text gives 0; an empty hypothesis gives 1 (all words deleted); “a b c d” against “d c b a” gives 1.0 (every word is wrong in place). WER can exceed 1 if the hypothesis has many insertions. It treats all errors as equal, so it says nothing about whether the *meaning* survived: “please don’t stop” heard as “please stop” is one error and a reversed meaning.

3.4.4 Video: a quick estimate

A video is a stack of frames. Treating each patch of each frame as a token, 16 frames of 224×224 with $P = 16$ give $16 \cdot 196 = 3136$ tokens, and $3136^2 = 9,834,496$ attention pairs: 256 times the cost of one image. Merging pairs of frames into one patch token (a “tubelet” of 2) halves the tokens to 1568 and cuts the attention cost by a factor of four. This is why video generators compress video aggressively (as latent diffusion does for images) and why long videos remain hard; I give only this arithmetic and did not run a video model.

3.5 Tier 5 (hardest): open questions

1. **How should guidance be chosen?** Figure 4 shows the trade-off in a toy; for real image models the right w depends on the model, the prompt and the taste of the viewer. There is no single metric (FID, CLIP score and human judgement can disagree).
2. **How good are the embeddings?** CLIP-style models are sensitive to wording, to what the 400 million pairs happened to contain, and to biases in captions. Zero-shot accuracy on a few classes says little about the rest.
3. **Safety, copyright and provenance.** Generative image, voice and video models can produce realistic fakes; training data may include copyrighted or private material (Unit 1’s memorisation, Unit 9). Detecting generated media reliably is an open problem.
4. **Efficiency.** Distilling a 50-step sampler to a few steps without Figure 3’s collapse, and compressing video, are active research areas.

4. Apply

4.1 Worked problems

Problem 1 (forward noising). At $t = 300$, $\bar{\alpha}_t = 0.394$. For $x_0 = 2$, $x_t = \sqrt{0.394} \cdot 2 + \sqrt{0.606} \epsilon = 1.255 + 0.778 \epsilon$: mean 1.255, standard deviation 0.778.

Problem 2 (recovering x_0). If the network predicts ϵ perfectly, $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) / \sqrt{\bar{\alpha}_t} = 2$ exactly. If its prediction is off by 0.05 at $t = 999$, $\hat{x}_0$ is off by $0.05/0.0064 \approx 7.8$.

Problem 3 (guidance). $\epsilon_c = 0.4$, $\epsilon_u = 0.1$, $w = 3$: $\tilde{\epsilon} = 4 \cdot 0.4 - 3 \cdot 0.1 = 1.3$, which is $0.1 + 4 \cdot 0.3$: four times the conditional gap added to the unconditional prediction.

Problem 4 (contrastive loss floor). A batch of 32 with 6 balanced classes has about 5.3 same-class pairs per row; a perfect class-level model has probability $1/5.3$ on the true pair: loss $\ln 5.3 = 1.67$.

Problem 5 (tokens). A 672×672 image with $P = 14$: $48^2 = 2304$ tokens; its attention cost relative to 196 tokens is $(2304/196)^2 = 138$.

Problem 6 (audio). 12 s of audio with a 10 ms hop: $1 + \lfloor 12 \cdot 16000/160 \rfloor = 1201$ frames, each of 80 log-mel numbers.

4.2 Code and figures

Everything is in `codes/multimodal/multimodal_np/` (see its README). The trained toy model is in `data/`. Sampling:

```

den = M.load_denoiser("data/diffusion_toy_weights.npz")
x = M.sample_ddim(den, n=1000, steps=50, rng=rng, c=0, w=3.0)      # class 0, guidance 3
x = M.sample_ddpm(den, n=1000, rng=rng)                          # 1000 stochastic steps
M.sliced_wasserstein(x, reference, rng)                           # quality against the
    data

```

Contrastive training is `M.train_clip(X, Tt, k, steps, batch, tau, lr, rng)`; audio features are `M.log_mel(waveform)`; error rate is `M.wer(reference, hypothesis)`.

For the practical “**develop a simple multimodal application**”, open the Colab notebook `u7_multimodal_real_models.ipynb`: BLIP answers questions about an image, CLIP classifies zero-shot, Whisper transcribes your voice and the course’s `wer` scores it. It was **not executed** while writing this document; the three models were confirmed to exist on the Hugging Face hub.

Try it in the playground

The **Diffusion denoiser** runs the *actual network trained here* in your browser (a JavaScript port checked to give the same samples as the NumPy code): scrub the progress slider from noise to data, set the steps to 3, and raise the guidance. The **Tokens** widget counts tokens and attention cost for images, video and audio.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), uncertainty (5.5).

5.1 Limits

- $t \rightarrow 0$: $x_t \rightarrow x_0$, little noise to predict; $\bar{\alpha}_t \rightarrow 0$: $x_t \rightarrow \epsilon$, and the noise is trivially predictable (Section 3.2).
- DDIM with 1 step from pure noise returns $\hat{x}_0$ computed with a divisor of 0.0064: the output is dominated by the network’s error (Figure 3: 9.62 at 2 steps).
- $w \rightarrow 0$: the conditional model; $w \rightarrow \infty$: extrapolation far from the learned region.
- Patch size $P = H$: a single token per image: no spatial detail.

5.2 Which term dominates

For video, attention (N^2) dominates once N is large: 3136 tokens give 9.8×10^6 pairs against 3136 tokens for the feed-forward part, which grows only linearly (Unit 2: the crossover is $N = 6d$).

5.3 Sensitivity

Sample quality is most sensitive to the number of steps at *few* steps (Figure 3: 3 steps 3.18, 10 steps 0.133, 25 steps 0.096). Variety is most sensitive to w (29.1° to 11.2°). Zero-shot accuracy was nearly insensitive to the batch size above 4 and to the temperature in the toy.

5.4 Failure modes

- **Too few sampling steps** (worse than noise).
- **Too much guidance** (collapse of variety, off-manifold artifacts).
- **Contrastive models matching words, not meaning**, and inheriting caption biases.
- **Hallucination about images** by vision-language models (Unit 5’s lesson applies).
- **Token explosion** for high-resolution images and long video.
- **Word error rate hiding meaning errors**.

5.5 Uncertainty

All experiments are 2-D or toy-feature, one seed for the diffusion model (the contrastive results use 4 seeds). The sliced Wasserstein distance is a crude quality measure and I give three reference values to calibrate it. Differences below about 0.01 between samplers in Figure 3 are within what a different random seed would change.

6. Evaluate

Diffusion versus the families of Unit 1. Diffusion models train stably and cover the data distribution well, at the price of many network evaluations per sample (Figure 3); GANs are fast but harder to train; autoregressive models give exact likelihoods but sample sequentially. Latent diffusion is the practical compromise for images. The ranking changes with time and task.

Contrastive versus generative training. CLIP-style models do not generate; they *align*. They are the bridge between text and images that conditioned diffusion models and vision-language models use.

When not to use. Diffusion for tasks that need exact, verifiable output (text in images, precise counts); zero-shot classification for high-stakes decisions without validation on your data; WER as the only measure of a speech system.

Wider context. Multimodal generation raises questions that text did not: realistic forgeries of faces and voices, consent for training data, and attribution. These are social as much as technical, and Unit 9 treats responsible use.

7. Create

What you can build from this

1. **Your own diffusion toy:** train `train_denoiser` on a spiral or on two moons and reproduce Figures 1 to 4.
2. **Guidance study:** make two overlapping classes and find whether guidance then improves purity.
3. **Step-count calculator:** for your sampler, find the smallest number of DDIM steps whose sliced Wasserstein distance is within 10% of 1000-step DDPM.
4. **Zero-shot tester:** train the toy CLIP with 30 classes and re-measure the effect of batch size.
5. **Token budget planner:** given a context window (Unit 4), how many images of a given size fit beside the text?

6. **Combine with Unit 5:** retrieve images by embedding the text query with the toy CLIP and ranking by cosine similarity.

8. Visualization lesson

charts taught here. How to read a spectrogram: time runs left to right, frequency bottom to top, brightness is energy; a rising line is a rising pitch (the chirp), a bright horizontal bar is a steady tone (the burst).

Chart type	Best for	How it misleads	matplotlib call
small multiples of scatter plots over time	a process that evolves (Fig. 1, 2)	fixed axes are essential or the cloud seems to move	<code>ax.scatter(...)</code> in a row with identical <code>xlim</code> , <code>ylim</code>
log-log line with reference lines	quality against steps with a floor and a noise level (Fig. 3)	log axes exaggerate small values near the floor	<code>ax.loglog(...)</code> ; <code>ax.axhline(...)</code>
twin-axis line plot	two effects of one setting (Fig. 4, right)	the two scales can be stretched to suggest a link	<code>ax.twinx()</code>
similarity-matrix heatmap	alignment of two sets (Fig. 5)	a diverging colour scale needs a symmetric limit	<code>ax.imshow(S, cmap="RdBu_r", vmin=-1, vmax=1)</code>
spectrogram image	time-frequency content (Fig. 7)	dB colour limits hide quiet parts	<code>ax.imshow(10*np.log10(P.T), origin="lower", aspect="auto")</code>

9. Problem set

Graded exercises

1. (easy) With $\bar{\alpha}_t = 0.0256$, how much of the original signal amplitude remains?
2. (medium) Two noisy steps: show that $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1\alpha_2$.
3. (medium-hard) Show that the guided prediction (5) equals $\epsilon_u + (1 + w)(\epsilon_c - \epsilon_u)$, and compute it for $\epsilon_c = 0.4$, $\epsilon_u = 0.1$, $w = 5$.
4. (hard) DDIM jumps in one step from $t = 999$ to $t' = 0$. Write the output in terms of the network's prediction and show that an error δ in ϵ_θ becomes an error of $\delta\sqrt{1 - \bar{\alpha}_{999}}/\sqrt{\bar{\alpha}_{999}}$ in $\hat{x}_0$. Evaluate it for $\delta = 0.1$.
5. (hardest) A video of 32 frames at 448×448 with $P = 16$ and a tubelet of 4. Compute the number of tokens, the attention pairs and the cost relative to one 224-pixel image of 196 tokens.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. Input: the noisy sample x_t , the step t (and a condition); output: the noise ϵ that was added (same shape as x_t).
2. DDIM is deterministic and the network predicts $\hat{x}_0$ at any step, so a subsequence of steps can be used; the trade-off is shown in Figure 3 (25 steps are enough on the toy, 3 are not).
3. $w = 0$ gives the plain conditional prediction $\epsilon_\theta(x, c)$.
4. It embeds the class names as text prompts and picks the prompt most similar to the image embedding (zero-shot classification).
5. Tokens grow $4\times$ and the attention cost $16\times$.

Explain it to a friend.

- Why do we train a network to predict *noise* rather than the clean image?
- Why does turning up guidance make all the pictures look alike?
- Why is one second of speech “100 frames”, and why is a video far more expensive than a photo?

Re-derive tomorrow without looking: (a) (1) from two steps; (b) the DDPM mean from the posterior mean; (c) the number of tokens and the attention ratio for an image of a given side and patch size.

11. Where next

- **Next:** Unit 8 (LLM agents and tool use), which combines everything so far (prompting, retrieval, tool calling) into loops that plan and act.
- **Terms met but not taught:** FID, vocoders and audio codecs, video diffusion, consistency and distillation methods.
- **Open threads:** how guidance behaves when classes overlap (Section 3.5); whether the Colab notebook reproduces the toy-model trends on real data.